

FLUJO COMPLETO: GENERACIÓN IA + DROPSHIPPING PICTUREM

RESUMEN DEL PROCESO

Este documento explica el flujo completo desde que un cliente genera una imagen con IA hasta que recibe su producto físico impreso.

FLUJO PASO A PASO

PASO 1: GENERACIÓN DE DISEÑO CON IA

Endpoint: POST /api/ai-wallpaper/generate

¿Qué hace?

- Cliente describe su diseño ideal en texto (prompt)
- Sistema usa OpenAI DALL-E 3 para generar imagen
- Si OpenAI API no está configurada, usa imágenes de Unsplash como fallback
- Retorna URL de la imagen generada

Código relevante:

```
// app/api/ai-wallpaper/generate/route.ts
export async function POST(req: NextRequest) {
  const { prompt, style, colors, width, height } = await req.json();

  if (openaiApiKey && openaiApiKey.startsWith('sk-')) {
    // Generar con OpenAI DALL-E 3
    const response = await openai.images.generate({
      model: "dall-e-3",
      prompt: enhancedPrompt,
      n: 1,
      size: "1024x1024",
      quality: "hd"
    });
  }

  return { imageUrl: response.data[0].url };
} else {
  // Fallback: Unsplash
  return { imageUrl: unsplashImages[randomIndex] };
}
```

Variables de entorno requeridas:

```
OPENAI_API_KEY=sk-... # Tu API key de OpenAI
```

PASO 2: CONFIGURACIÓN DE PRODUCTO

Endpoint: POST /api/pictorem/pricing

¿Qué hace?

- Cliente elige material (canvas, metal, acrylic, paper)
- Cliente elige tamaño (width x height en pulgadas)
- Cliente elige tipo de producto (stretched, roll, etc.)
- Sistema calcula precio usando API de Pictorem
- Si API falla, usa pricing de fallback basado en área
- Aplica markup para ganancia

Ejemplo de precios:

Materiales:

- Canvas: \$0.12 por pulgada cuadrada
- Metal: \$0.25 por pulgada cuadrada
- Acrílico: \$0.30 por pulgada cuadrada
- Papel: \$0.08 por pulgada cuadrada

Markup: 2.0x (200% sobre costo de Pictorem)

Precio mínimo: \$25

Precio máximo: \$500

Código relevante:

```
// app/api/pictorem/pricing/route.ts
const pictoremCost = area * basePricePerSqIn * numCopies;
const customerPrice = Math.max(
  minPrice,
  Math.min(maxPrice, pictoremCost * baseMarkup)
);
const profit = customerPrice - pictoremCost;

return {
  customerPrice,
  pictoremCost,
  profit,
  estimatedDeliveryDays: 7-10
};
```

Variables de entorno requeridas:

```
PICTOREM_API_KEY=tu_api_key_aqui
PICTOREM_SHOP_CODE=tu_shop_code_aqui
PICTOREM_BASE_URL=https://www.pictorem.com/artflow
```

PASO 3: CREACIÓN DE ORDEN

Endpoint: POST /api/ai-wallpaper/create-order

¿Qué hace?

- Crea orden en base de datos con estado PENDING
- Guarda todos los detalles:

- Imagen generada (URL)
- Configuración de producto (material, tamaño, etc.)
- Info del cliente (nombre, email, teléfono)
- Dirección de envío
- Precios (costo Pictorem, precio cliente, ganancia)
- Genera código de preorden para Pictorem
- Retorna `orderId` para el pago

Código de preorden Pictorem:

```
// Formato: numCopies|material|type|orientation|width|height|additional
// Ejemplo: 1|canvas|stretched|horizontal|24|36|regular|bordercolor|c15|none|none|
none|opt

const preorderCode = [
  numCopies,
  material,
  type,
  orientation,
  width,
  height,
  ...additional
].join('|');
```

Estados de orden:

- `PENDING` - Orden creada, esperando pago
- `CONFIRMED` - Pago confirmado
- `PROCESSING` - Enviada a Pictorem
- `SHIPPED` - Enviada al cliente
- `DELIVERED` - Entregada
- `CANCELLED` - Cancelada

Estados de Al Order:

- `READY_FOR_PAYMENT` - Lista para pago
- `PAID` - Pagada
- `SENT_TO_PICTOREM` - Enviada a Pictorem
- `ERROR` - Error al enviar

PASO 4: PROCESO DE PAGO

Opciones de pago:

1. **Stripe** - Tarjetas de crédito/débito
2. **PayPal** - Cuenta PayPal

Flujo con Stripe:

Endpoint: `POST /api/payments/stripe/create-session`

```

const session = await stripe.checkout.sessions.create({
  mode: 'payment',
  line_items: [{
    price_data: {
      currency: 'usd',
      product_data: {
        name: 'AI Generated Wallpaper',
        description: aiOrder.prompt,
        images: [aiOrder.generatedImageUrl]
      },
      unit_amount: Math.round(order.total * 100)
    },
    quantity: 1
  }],
  metadata: { orderId: order.id },
  success_url: successUrl,
  cancel_url: cancelUrl
});

// Redirige a Stripe Checkout
return { url: session.url };

```

Variables de entorno requeridas:

```

STRIPE_SECRET_KEY=sk_test... # 0 sk_live... para producción
STRIPE_PUBLISHABLE_KEY=pk_test... # 0 pk_live...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test... # Para cliente
STRIPE_WEBHOOK_SECRET=whsec... # Para verificar webhooks

```

Flujo con PayPal:

Endpoint: POST /api/payments/paypal/create-order

```

const order = await paypal.orders.create({
  intent: 'CAPTURE',
  purchase_units: [{
    amount: {
      currency_code: 'USD',
      value: orderTotal.toFixed(2)
    },
    description: aiOrder.prompt
  }],
  application_context: {
    return_url: successUrl,
    cancel_url: cancelUrl
  }
});

// Redirige a PayPal
return { approvalUrl: order.links[1].href };

```

Variables de entorno requeridas:

```

PAYPAL_CLIENT_ID=tu_client_id
PAYPAL_CLIENT_SECRET=tu_client_secret
PAYPAL_MODE=sandbox # 0 'live' para producción

```

PASO 5: WEBHOOK DE PAGO (AUTOMÁTICO)

Endpoint: POST /api/payments/stripe/webhook

¿Qué hace Stripe?

- Cuando el pago se completa, Stripe envía un webhook
- Webhook valida firma para seguridad
- Procesa evento `checkout.session.completed`

Acciones automáticas:

```
async function handleCheckoutCompleted(session) {  
  // 1. Actualizar transacción de pago  
  await prisma.paymentTransaction.update({  
    where: { sessionId: session.id },  
    data: {  
      status: 'COMPLETED',  
      transactionId: session.payment_intent  
    }  
  });  
  
  // 2. Actualizar orden  
  await prisma.order.update({  
    where: { id: orderId },  
    data: {  
      status: 'CONFIRMED',  
      stripePaymentId: session.payment_intent  
    }  
  });  
  
  // 3. Actualizar AI order  
  await prisma.aiWallpaperOrder.update({  
    where: { orderId },  
    data: { status: 'PAID' }  
  });  
  
  // 4. 🚀 ENVIAR A PICTUREM (paso crítico)  
  await sendOrderToPictorem(orderId);  
}
```

PASO 6: ENVÍO A PICTUREM (AUTOMÁTICO)

Función: `sendOrderToPictorem(orderId)`

¿Qué hace?

- Obtiene orden y AI order de base de datos
- Prepara información de envío
- Prepara items de orden con imagen generada
- Llama a Pictorem API para crear orden física
- Actualiza orden con ID de Pictorem

Código relevante:

```

async function sendOrderToPictorem(orderId: string) {
  // 1. Obtener orden
  const order = await prisma.order.findUnique({
    where: { id: orderId },
    include: { aiWallpaperOrders: true }
  });

  const aiOrder = order.aiWallpaperOrders[0];

  // 2. Preparar info de envío
  const deliveryInfo = {
    firstname: order.shippingName.split(' ')[0],
    lastname: order.shippingName.split(' ').slice(1).join(' '),
    address1: order.shippingAddress1,
    address2: order.shippingAddress2 || '',
    city: order.shippingCity,
    province: order.shippingState,
    country: order.shippingCountry,
    cp: order.shippingZip,
    phone: order.shippingPhone
  };

  // 3. Preparar items
  const orderItems = [{
    code: aiOrder.pictoremPreorderCode,
    fileurl: aiOrder.generatedImageUrl, // ⚠ Debe ser URL pública
    filetype: 'jpg',
    bordercolorhex: aiOrder.borderColor || 'ffffff',
    thanknotemsg: 'Custom design by Barrera Wallpaper'
  }];

  // 4. 🚀 Enviar a Pictorem
  const response = await pictoremClient.sendOrder(
    deliveryInfo,
    orderItems,
    `Barrera Wallpaper Order ${order.orderNumber}`
  );

  // 5. Actualizar con ID de Pictorem
  if (response.status && response.orderid) {
    await prisma.order.update({
      where: { id: orderId },
      data: {
        pictoremOrderId: response.orderid,
        pictoremStatus: 'confirmed',
        status: 'PROCESSING'
      }
    });

    await prisma.aiWallpaperOrder.update({
      where: { orderId },
      data: {
        status: 'SENT_TO_PICTOREM',
        pictoremOrderId: response.orderid
      }
    });
  }
}

```

⚠ REQUISITO CRÍTICO:

La imagen generada debe estar en una URL pública accesible:

-  URLs de OpenAI (válidas por 1 hora)
-  URLs de tu servidor público
-  URLs de CDN (AWS S3, Cloudflare, etc.)
-  URLs de localhost
-  URLs privadas

PASO 7: PROCESAMIENTO EN PICTUREM

¿Qué hace Pictorem?

1. **Recibe orden** con código de preorden e imagen
2. **Descarga imagen** desde URL pública
3. **Imprime** en material seleccionado
4. **Empaca** el producto
5. **Envía** al cliente
6. **Actualiza tracking** (número de rastreo)

Estados en Pictorem:

- `confirmed` - Orden confirmada
- `in_production` - En producción
- `shipped` - Enviada
- `delivered` - Entregada

Tiempo estimado:

- Canvas: 7-10 días hábiles
- Metal: 10-14 días hábiles
- Acrílico: 10-14 días hábiles
- Papel: 5-7 días hábiles

PASO 8: SEGUIMIENTO Y REINTENTOS

Ver órdenes de Pictorem (Admin):

Endpoint: `GET /api/pictorem/sync-order`

```
// Lista todas las órdenes enviadas a Pictorem
const orders = await prisma.order.findMany({
  where: { pictoremOrderId: { not: null } },
  include: { aiWallpaperOrders: true }
});
```

Reintentar orden fallida (Admin):

Endpoint: `POST /api/pictorem/retry-order`

¿Cuándo usar?

- Orden tiene estado `ERROR`
- Pictorem rechazó la orden
- URL de imagen expiró
- Problemas de conexión

```
// Reintenta enviar orden a Pictorem
await fetch('/api/pictorem/retry-order', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ orderId: 'order_id_aqui' })
});
```

CONFIGURACIÓN COMPLETA

1. Variables de Entorno (.env)

```
# Base de Datos
DATABASE_URL=mysql://usuario:password@localhost:3306/barrera_db

# Autenticación
NEXTAUTH_SECRET=tu_secret_aleatorio_32_caracteres
NEXTAUTH_URL=https://barrerawallpaper.com

# OpenAI (Generación de Imágenes)
OPENAI_API_KEY=sk-proj-... # Obtener en https://platform.openai.com/api-keys

# Pictorem (Dropshipping)
PICTOREM_API_KEY=tu_api_key # Contactar soporte de Pictorem
PICTOREM_SHOP_CODE=tu_shop_code
PICTOREM_BASE_URL=https://www.pictorem.com/artflow

# Stripe (Pagos)
STRIPE_SECRET_KEY=sk_test_... # De dashboard.stripe.com
STRIPE_PUBLISHABLE_KEY=pk_test_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_...
STRIPE_WEBHOOK_SECRET=whsec_... # Crear en Stripe Dashboard

# PayPal (Pagos alternativos)
PAYPAL_CLIENT_ID=tu_client_id # De developer.paypal.com
PAYPAL_CLIENT_SECRET=tu_client_secret
PAYPAL_MODE=sandbox # 0 'live' en producción

# AWS S3 (Almacenamiento - Opcional)
AWS_BUCKET_NAME=tu_bucket
AWS_FOLDER_PREFIX=barrera/
```

2. Cómo Obtener API Keys

OpenAI:

1. Ir a <https://platform.openai.com>
2. Crear cuenta o iniciar sesión
3. Ir a “API Keys”
4. Click “Create new secret key”
5. Copiar key (empieza con `sk-proj-...`)
6. **Agregar créditos:** Billing → Add payment method

Costo:

- DALL-E 3 HD 1024x1024: ~\$0.08 por imagen
- DALL-E 3 Standard: ~\$0.04 por imagen

Pictorem:

1. Ir a <https://www.pictorem.com>
2. Contactar ventas/soporte
3. Solicitar cuenta de dropshipping
4. Obtener:
 - API Key (artFlowKey)
 - Shop Code
 - Acceso al dashboard

Pricing:

- No hay costo de setup
- Pagas por impresión (precio mayorista)
- Tú defines tu markup

Stripe:

1. Ir a <https://dashboard.stripe.com>
2. Crear cuenta
3. Activar pagos
4. Obtener keys:
 - Developers → API Keys
 - Test mode: `sk_test_...` y `pk_test_...`
 - Production: Activar cuenta y obtener `sk_live_...`
5. Configurar webhook:
 - Developers → Webhooks → Add endpoint
 - URL: `https://barrerawallpaper.com/api/payments/stripe/webhook`
 - Eventos: `checkout.session.completed`, `payment_intent.succeeded`, `payment_intent.payment_failed`

Costo:

- 2.9% + \$0.30 por transacción exitosa

PayPal:

1. Ir a <https://developer.paypal.com>
2. Crear cuenta de desarrollador
3. Dashboard → Apps → Create App
4. Obtener Client ID y Secret
5. Sandbox para pruebas
6. Live credentials para producción

Costo:

- 2.99% + \$0.49 por transacción
-

CÓMO PROBAR EL FLUJO COMPLETO

Modo de Prueba (Sin APIs Reales):

```
# .env para pruebas locales
OPENAI_API_KEY= # Dejar vacío para usar Unsplash
PICTOREM_API_KEY=test_key # Usará pricing de fallback
STRIPE_SECRET_KEY= # Dejar vacío para simular pago
```

Con esto:

- ☒ Generación de imagen usará Unsplash
- ☒ Pricing usará cálculos de fallback
- ☒ Orden se creará en DB
- ☒ No se enviará a Pictorem (falta API key real)

Modo de Producción:

```
# .env para producción
OPENAI_API_KEY=sk-proj-real-key # API real con créditos
PICTOREM_API_KEY=real_api_key # API real de Pictorem
STRIPE_SECRET_KEY=sk_live... # Stripe en modo live
STRIPE_WEBHOOK_SECRET=whsec... # Webhook configurado
```

Con esto:

- ☒ Generación de imagen usará DALL-E 3
- ☒ Pricing consultará Pictorem API real
- ☒ Pagos reales con Stripe
- ☒ Órdenes se enviarán a Pictorem
- ☒ Cliente recibirá producto físico



PROBLEMAS COMUNES Y SOLUCIONES

Problema 1: Imagen no se genera

Síntomas:

- Error "OpenAI API key is invalid"
- Devuelve imagen de Unsplash

Soluciones:

1. Verificar que `OPENAI_API_KEY` esté configurada
2. Verificar que key empiece con `sk-proj-`
3. Verificar que cuenta tenga créditos
4. Probar key: <https://platform.openai.com/playground>

Problema 2: Pricing no funciona

Síntomas:

- Error "Failed to calculate pricing"
- Precio siempre es \$0

Soluciones:

1. Verificar `PICTOREM_API_KEY` configurada
 2. Verificar formato de preorder code
 3. Sistema usará fallback si API falla (normal)
 4. Revisar logs: `pm2 logs barrera-wallpaper`
-

Problema 3: Orden no se crea**Síntomas:**

- Error "Failed to create order"
- Base de datos no accesible

Soluciones:

1. Verificar `DATABASE_URL` correcta
 2. Ejecutar migraciones: `yarn prisma migrate deploy`
 3. Verificar conexión a MySQL: `pm2 logs`
-

Problema 4: Pago no procesa**Síntomas:**

- Redirige pero no completa
- Webhook no se ejecuta

Soluciones:**Stripe:**

1. Verificar `STRIPE_SECRET_KEY` correcta
2. Usar tarjeta de prueba: `4242 4242 4242 4242`
3. Verificar webhook configurado en Stripe
4. URL webhook: `https://tudominio.com/api/payments/stripe/webhook`
5. Verificar `STRIPE_WEBHOOK_SECRET`

PayPal:

1. Usar cuenta sandbox para pruebas
 2. Verificar `PAYPAL_MODE=sandbox`
 3. Probar con cuenta sandbox de prueba
-

Problema 5: Orden no se envía a Pictorem**Síntomas:**

- Estado queda en `PAID`
- No aparece `pictoremOrderId`
- Estado del AI order: `ERROR`

Causas comunes:

1. **URL de imagen expiró** (OpenAI URLs expiran en 1 hora)
2. **Pictorem no puede acceder a la imagen** (URL privada)
3. **Código de preorden inválido**
4. **Credenciales de Pictorem incorrectas**

Soluciones:**Para URL expirada:**

```
// Opción 1: Descargar y subir a tu servidor
const imageBuffer = await fetch(openaiUrl).then(r => r.buffer());
const publicUrl = await uploadToS3(imageBuffer);

// Opción 2: Usar AI order con imagen almacenada
await prisma.aiWallpaperOrder.update({
  where: { id: aiOrderId },
  data: { generatedImageUrl: publicUrl }
});
```

Para reintentar manualmente:

```
# Como admin, ir a panel
/admin/orders

# Click en "Retry" para órdenes con ERROR
# O usar API:
curl -X POST https://barrerawallpaper.com/api/pictorem/retry-order \
-H "Content-Type: application/json" \
-d '{"orderId": "order_id_aqui"}'
```



TRACKING Y MONITOREO

Dashboard de Admin:**URL:** /admin/orders**Muestra:**

- Lista de todas las órdenes
- Estado de cada orden
- Órdenes enviadas a Pictorem
- Órdenes con errores
- Botón "Retry" para reintentar

Logs de PM2:

```
# Ver logs en tiempo real
pm2 logs barrera-wallpaper

# Ver solo errores
pm2 logs barrera-wallpaper --err

# Ver últimas 100 líneas
pm2 logs barrera-wallpaper --lines 100
```

Buscar problemas:

```
# Errores de Pictorem
pm2 logs barrera-wallpaper | grep "Pictorem"

# Errores de OpenAI
pm2 logs barrera-wallpaper | grep "OpenAI"

# Errores de pago
pm2 logs barrera-wallpaper | grep "payment"
```

COSTOS Y MÁRGENES

Costos por Orden:

Generación IA (OpenAI DALL-E 3):

- Standard 1024x1024: \$0.04
- HD 1024x1024: \$0.08

Impresión Pictorem (ejemplo Canvas 24x36):

- Área: 864 pulgadas cuadradas
- Costo: $864 \times \$0.12 = \103.68
- Envío: Incluido en precio Pictorem

Procesamiento de pago (Stripe):

- Tarifa: 2.9% + \$0.30
- Para \$200: $\$5.80 + \$0.30 = \$6.10$

Costo total: $\$0.08 + \$103.68 + \$6.10 = \109.86

Precio al Cliente:

Costo Pictorem: \$103.68
Markup: 2.0x
Precio base: \$207.36

Precio final al cliente: \$210.00
Ganancia bruta: $\$210.00 - \$109.86 = \$100.14$
Margen: 47.7%

Ejemplo de Pricing:

Material	Tamaño	Área (sq in)	Costo Pictorem	Markup 2x	Precio Cliente	Ganancia
Canvas	18x24	432	\$51.84	\$103.68	\$105.00	\$53.16
Canvas	24x36	864	\$103.68	\$207.36	\$210.00	\$106.32
Metal	16x20	320	\$80.00	\$160.00	\$165.00	\$85.00
Metal	24x36	864	\$216.00	\$432.00	\$435.00	\$219.00
Acrylic	16x20	320	\$96.00	\$192.00	\$195.00	\$99.00
Paper	18x24	432	\$34.56	\$69.12	\$75.00	\$40.44

Nota: Precios aproximados. Los precios reales de Pictorem varían según:

- Material y tipo específico
- Opciones adicionales (marco, borde, etc.)
- Cantidad
- Destino de envío



CHECKLIST DE CONFIGURACIÓN

Configuración Inicial:

- ☐ Base de datos MySQL creada y configurada
- ☐ Variables de entorno configuradas en Hostinger
- ☐ Migraciones ejecutadas (`yarn prisma migrate deploy`)
- ☐ Base de datos sembrada con datos iniciales
- ☐ PM2 corriendo sin errores

Integración OpenAI:

- ☐ Cuenta de OpenAI creada
- ☐ API Key generada
- ☐ Créditos agregados a la cuenta
- ☐ Variable `OPENAI_API_KEY` configurada
- ☐ Probado generación de imagen en AI Studio

Integración Pictorem:

- ☐ Cuenta de dropshipping solicitada
- ☐ API Key recibida
- ☐ Shop Code configurado
- ☐ Variables `PICTOREM_*` configuradas
- ☐ Probado cálculo de pricing

Integración Stripe:

- ☐ Cuenta de Stripe creada
- ☐ Test mode activado
- ☐ API Keys copiadas (sk_test_... y pk_test_...)
- ☐ Variables `STRIPE_*` configuradas
- ☐ Webhook endpoint creado en Stripe
- ☐ Webhook secret configurado
- ☐ Probado pago con tarjeta de prueba

Integración PayPal (Opcional):

- ☐ Cuenta de PayPal Developer creada
- ☐ App de Sandbox creada
- ☐ Client ID y Secret obtenidos
- ☐ Variables `PAYPAL_*` configuradas
- ☐ Probado pago con cuenta sandbox

Pruebas End-to-End:

- ☐ Generación de imagen funciona
- ☐ Configuración de producto funciona
- ☐ Cálculo de pricing correcto
- ☐ Creación de orden exitosa
- ☐ Pago procesa correctamente
- ☐ Webhook recibido y procesado
- ☐ Orden enviada a Pictorem automáticamente
- ☐ Estado actualizado a `PROCESSING`
- ☐ `pictoremOrderId` presente en orden

Dashboard de Admin:

- ☐ Login como admin funciona
- ☐ Lista de órdenes visible
- ☐ Órdenes AI filtradas correctamente
- ☐ Botón “Retry” funciona para órdenes con error
- ☐ Sync de órdenes funciona

SOPORTE

Problemas con OpenAI:

- Documentación: <https://platform.openai.com/docs>
- Soporte: <https://help.openai.com>

Problemas con Pictorem:

- Website: <https://www.pictorem.com>
- Email: info@pictorem.com
- Documentación API: (proporcionada al activar cuenta)

Problemas con Stripe:

- Documentación: <https://stripe.com/docs>
- Soporte: <https://support.stripe.com>

Problemas con PayPal:

- Developer Portal: <https://developer.paypal.com>
- Soporte: <https://www.paypal.com/us/smarthelp/contact-us>



PRÓXIMOS PASOS

1. **Configurar todas las variables de entorno**
 2. **Obtener API keys de OpenAI y Pictorem**
 3. **Configurar Stripe webhook**
 4. **Probar flujo completo en modo test**
 5. **Activar modo producción**
 6. **Monitorear primeras órdenes**
-

Última actualización: 2 de enero de 2025

Versión: 1.0